

The Architect's 20%: Four IAM Trust Assumptions That Unlock 80% of Access Control Failures

As a security architect, you don't design IAM by configuring SSO portals—you **map where identity systems assume trust** and design controls that isolate those assumptions. These four concepts explain why 90% of breaches involve identity compromise, why "least privilege" fails in practice, and why federation creates blast-radius explosions. Each includes a 15-minute lab using only free-tier cloud accounts (AWS/Azure) or public identity providers—no enterprise infrastructure required.

Concept 1: Identity as a *Claim* (Not a Fact)

Why it unlocks 80%: Identities aren't "real people"—they're *claims issued by a trust anchor* (IdP). Breach the anchor (Okta, Azure AD), and *all downstream systems trust the forged claims*. This explains why identity providers are the #1 target in modern breaches.

15-Minute Lab: *Witness Identity as a Claim Chain*

```
# Step 1: Get a real identity token (Google as IdP)
# Go to https://developers.google.com/oauthplayground
# Select "Google OAuth2 API v2" → Authorize APIs → Click "Authorize APIs"
# Copy the resulting "Access token" and "ID token"

# Step 2: Decode the ID token (JWT) to see claims
# Paste token at https://jwt.io
# Observe claims: "sub" (subject), "email", "iss" (issuer=accounts.google.com)

# Step 3: Simulate claim forgery (harmless demo)
# In jwt.io, change "email" to "attacker@evil.com" → See signature INVALID
# Now imagine attacker compromises Google's signing key → signature VALID
```

Interpretation: Your "identity" is just a signed JSON blob. The resource server (e.g., Gmail) *trusts the signature*, not the user. If the signing key leaks (Okta 2022 breach), every claim becomes forgeable.

Architectural implication: Never treat identity providers as "trusted infrastructure"—treat them as **high-value attack surfaces requiring isolation**. Implement *identity blast radius containment*: separate production/non-production IdPs, enforce step-up authentication for

sensitive actions, and monitor for anomalous claim patterns (e.g., user suddenly accessing EU resources from Nigeria).

Underexplored perspective most architects miss:

Identity claims have **implicit trust transitivity** that creates hidden blast radii. Example: Your SSO provider (Okta) trusts Google as an IdP → Google trusts Facebook for social login → Facebook gets breached → attackers forge claims that Okta *and all your apps* accept. Architects who map *direct* trust relationships miss **transitive trust chains** spanning 3+ providers. The real control isn't "use MFA"—it's **trust boundary segmentation**: break transitive chains via *identity domains* (e.g., production apps trust ONLY Azure AD, never social logins).

Concept 2: Authorization Boundaries ≠ Authentication Boundaries

Why it unlocks 80%: Authentication proves *who you are*; authorization decides *what you can do*. These boundaries are *deliberately separated* in modern systems (OAuth scopes, AWS IAM conditions)—but attackers exploit the gap between them. This explains token hijacking, confused deputy attacks, and why MFA alone doesn't prevent data exfiltration.

15-Minute Lab: Map the AuthN/AuthZ Gap

```
# Step 1: Authenticate to GitHub (AuthN)
# Go to https://github.com/settings/tokens?type=beta
# Create a fine-grained token with ONLY "public_repo" scope

# Step 2: Attempt unauthorized action (AuthZ enforcement)
curl -H "Authorization: token YOUR_TOKEN" https://api.github.com/user/repos
# Works: token has "public_repo" scope

curl -H "Authorization: token YOUR_TOKEN" https://api.github.com/user/repos -X POST -d '{}'
# Fails: 403 Forbidden → AuthZ boundary enforced despite valid AuthN

# Step 3: Simulate confused deputy (harmless demo)
# Imagine a CI/CD service with YOUR token gets tricked into deleting prod repos
# The token is *valid* (AuthN passed) but used in unintended context (AuthZ bypassed)
```

Interpretation: Your token passed authentication—but GitHub's API *re-evaluated authorization per request* using scopes. The gap: tokens often carry *excessive scopes* ("admin" instead of "read-only"), letting attackers abuse valid tokens for unintended actions.

Architectural implication: Design **context-aware authorization**: never rely on token validity alone. Enforce *runtime constraints* (AWS IAM conditions: `aws:SourceIp`, `aws:MultiFactorAuthPresent`), and implement *just-in-time access* (Azure PIM) that shortens the AuthN → AuthZ window.

🔒 Underexplored perspective most architects miss:

Authorization boundaries suffer from **ambient authority**—where processes inherit privileges from their environment rather than explicit grants. Example: An AWS Lambda with `s3:*` permissions runs code that *should* only read one bucket—but a dependency library with a path traversal bug writes to `s3://prod-backups`. Architects who design *static* RBAC policies miss that **runtime context determines actual privilege**. The real control isn't "narrow roles"—it's **capability-based security**: grant tokens with *explicit resource constraints* (e.g., AWS STS `AssumeRole` with `Policy` parameter limiting to single bucket).

🔑 Concept 3: Token Delegation Chains — "Trust Flows Downward Like DNS"

Why it unlocks 80%: OAuth 2.0 and SAML create *delegation chains* where Service A trusts IdP, Service B trusts Service A's token, etc. Break one link (compromise a service account), and you own all downstream services. This explains supply chain attacks (SolarWinds) and why workload identities are critical attack vectors.

🔧 15-Minute Lab: Trace a Delegation Chain

```
# Step 1: Create AWS IAM role trust relationship (free tier)
# In AWS Console: IAM → Roles → Create role
# Trusted entity: "Another AWS account" (use your own account ID)
# Attach policy: AmazonS3ReadOnlyAccess

# Step 2: Simulate delegation chain
# Account A (IdP) issues token → Account B (Service) assumes role → Accesses S3
aws sts assume-role --role-arn arn:aws:iam::YOUR_ACCOUNT:role/TestRole --role-session-name TestSession

# Step 3: Observe transitive trust
# The returned credentials work for S3 *without* Account B having direct S3 permissions
# Trust flowed: Account A → Account B → S3
```

Interpretation: The role's trust policy (`Trusted entities`) defines *who can delegate*. If Account A gets compromised, attackers forge tokens that Account B accepts—bypassing Account B's own security controls.

Architectural implication: Delegation chains require **short-lived tokens** (AWS STS default: 1 hour) and **explicit trust boundaries** (never use `Principal: "*"`). Monitor for anomalous delegation patterns (e.g., service account suddenly accessing HR data).

Underexplored perspective most architects miss:

Delegation chains have **implicit elevation points** where tokens gain privileges *without user interaction*. Example: Azure AD application permissions (`Application.ReadWrite.All`) let a compromised service principal *grant itself* new permissions via Microsoft Graph API—creating self-elevating tokens. Architects who audit *initial* token grants miss that **tokens can mutate their own privileges** via API calls. The real control isn't "limit scopes"—it's **token immutability enforcement**: use short-lived tokens + deny self-modification permissions (e.g., AWS IAM `Deny` on `sts:AssumeRole` for roles that shouldn't delegate further).

Concept 4: Privilege as a *Time-Bound Resource* (Not a Static Attribute)

Why it unlocks 80%: Privilege isn't "user has admin role"—it's *temporary access granted under specific conditions*. Static RBAC fails because humans forget to revoke access (leavers), and attackers abuse standing privileges. This explains why PIM/PAM exists and why "just-in-time" access reduces breach impact.

15-Minute Lab: *Measure Your Standing Privilege Blast Radius*

```
# Step 1: Audit standing privileges in free-tier Azure
# Go to https://portal.azure.com → Azure Active Directory → Roles and administrators
# Click "Global Administrator" → "Assignments"
# Count how many users have *permanent* Global Admin (not eligible via PIM)

# Step 2: Simulate breach impact
# If one permanent admin account gets phished → attacker owns entire tenant
# Compare to PIM model: attacker gets 4-hour window before MFA re-challenge

# Step 3: Quantify "toxic combinations" (free tool)
# Use Microsoft's Azure AD Access Reviews demo:
# https://entra.microsoft.com → Identity Governance → Access Reviews
# Create review for "Users with both Finance and HR access"
# Observe: Standing access creates toxic combos that JIT access prevents
```

Interpretation: Standing privileges = *always-available attack surface*. Just-in-time access converts privilege from a *static attribute* to a *time-bound resource*—reducing blast radius from "infinite" to "hours."

Architectural implication: Design **ephemeral privilege**: all sensitive access should require approval + MFA + time limit. Treat standing privileges as *technical debt* to be eliminated quarterly.

 **Underexplored perspective most architects miss:**
Privilege time-bounds create **temporal attack windows** that shift attacker economics. Example: An attacker who compromises a JIT-access account must *act within the 4-hour window* before access revokes—forcing them to automate exfiltration or risk detection. Architects who focus only on *granting* JIT access miss that **revocation reliability** is the true control point. If revocation fails (e.g., AWS STS token not invalidated on role deletion), JIT becomes theater. The real metric isn't "JIT implemented"—it's **mean time to revocation (MTTR)** measured via red team exercises.

Synthesis: How These Concepts Explain Real Breaches

Breach	Failed Trust Assumption	Architectural Failure
Okta 2022	Identity as a claim	Single IdP for all apps → transitive trust explosion
SolarWinds SUNBURST	Token delegation chains	Compromised build system issued trusted tokens to customers
Colonial Pipeline	AuthN/AuthZ boundary gap	Valid credentials used for unintended lateral movement
Twilio 2022	Standing privilege	SMS API keys with no time-bound constraints → \$10M fraud

The Architect's Mantra:

"IAM security isn't about stronger passwords—it's about isolating trust assumptions (claims, delegation, boundaries) and converting standing privilege into ephemeral access."

Your Architectural Foundation: The Four Leverage Points

Trust Assumption	What It Controls	Hardening Response	What Most Miss
Identity as claim	Entire identity blast radius	Segment identity domains; monitor claim anomalies	Transitive trust chains spanning 3+ providers
AuthN/AuthZ gap	Token abuse after compromise	Context-aware authZ (IP/MFA conditions); capability tokens	Ambient authority in runtime environments
Delegation chains	Supply chain compromise	Short-lived tokens; deny self-modification	Tokens that mutate their own privileges via APIs
Standing privilege	Lateral movement speed	JIT access + strict revocation SLAs	Revocation reliability > grant mechanics

Your Next Challenge (20 Minutes)

Build an "identity blast radius map" for one cloud service you use:

1. Pick a service (e.g., GitHub, AWS Console, Google Workspace)
2. Trace its trust chain:
 - What IdP issues your identity token? (Google? Azure AD?)
 - What downstream services trust that token? (List 3 apps)
 - What privileges does the token carry *by default*? (Check scopes/roles)
 - How long does the token remain valid without re-authentication?

3. Identify **one unnecessary trust link** (e.g., GitHub trusting Google for *all* scopes) and design a control to break it (e.g., "Use GitHub-native 2FA + limit OAuth scopes to **repo** only").

Deliverable: A 3-sentence architectural decision:

"I will [action] to break the trust link between [IdP] and [service] because [risk], reducing blast radius from [X] to [Y]."

Why This Is the True 20%

These four concepts explain:

- Why Okta breaches compromise 1000+ companies (identity as claim)
- Why MFA bypasses still work (AuthN/AuthZ gap)
- Why supply chain attacks succeed (delegation chains)
- Why standing privileges enable rapid lateral movement (time-bound privilege)

Master these, and you'll intuitively understand 80% of IAM failures—not by memorizing OAuth flows, but by **seeing the trust assumptions attackers exploit**. That's the architect's advantage: designing systems where identity trust is *minimal, time-bound, and non-transitive*—not just "MFA everywhere."